Lab2

Jacob Krol

2026-02-20

Lab2: Comparing Nonlinear Regressions

Due: 11:59PM Friday, February 27 on Canvas as a knitted pdf file of your team's Quarto document

1. You will individually fit **six** nonlinear regression models on a single X variable from your team's chosen dataset. You will use CV (including the built-in LOOCV functions in R) on at least three of these models to determine the best tuning parameters (e.g., λ , K number of knots or steps, or degrees of freedom). You will compare the six models, choose the best one, make a prediction at an important value of X, interpret the results, and comment on the ethics of the situation.
2. Teams will use the individual contributions to build a Generalized Additive Model (GAM), fit it, make predictions, make interpretations, show visualizations, etc.
3. Each individual will comment on how they contributed to their team's GAM.

Instructions for Lab2

In HW2, you applied several types of shrinkage and selection methods for linear models (shrinkage and selection methods) on a dataset of your choosing. This lab will be similar, but you will apply the nonlinear regression methods of Chapter 7 in *ISLR2* and use cross validation to choose the tuning parameters and select the best method. Individuals will model individual variables from the same dataset (one variable for each team member) and then combine their work into one team GAM that uses multiple X variables to predict Y.

What teams need to do first

1. Choose a dataset that has enough interesting X variables that you can combine to model/predict Y. You can use one of the datasets that a teammate used for HW2 or pick a new one. You cannot use the Wage data from *ISLR2*.

2. Decide who will model which X variable individually and when they need to be done to contribute to the team GAM.

What individuals need to do

1. Given your team's dataset and your X variable, fit a polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and a local regression (six models).
2. Use CV to determine the optimal tuning parameters for at least three of your models. Sometimes just choosing the tuning parameter (e.g., picking the number of cuts in X for a step function regression) based on your knowledge of Q1 and Q3 makes more sense than using CV.
3. Compare the CV MSE or other appropriate model summary (e.g., misclassification rate) for your models. Choose the best model method.
4. At an important or significant value of X (explain why that value of x0 is important or significant), use your model to predict Y at that value.
5. Interpret your prediction and generally what the model is telling you about Y.
6. Comment on any ethical implications of this work.
7. Optional: Create visualizations of your models or just of the best model.

What teams need to do

1. Create a GAM based on the work of your individual team members.
2. Visualize the GAM (R has some good built-in plotting functions, such as plot.gam()).
3. Choose a value of X that is important or interesting (explain why) and use that point to make a prediction for Y. Interpret this prediction.
4. How does the GAM compare with a linear method (e.g., lasso, ridge, PCR, etc.)?

Individual contributions to team GAM

Each individual must comment on their contributions to the team GAM. For example, write something like: 'I told person X that my results showed that the natural spline with 4 knots was the best model for variable x1. Then I created a visualization to help interpret the GAM.' Or, 'I used the GAM to make a prediction for the point x0.' Or, 'I didn't actually do anything for the team section.' Only individuals who contribute to the team section will get points for the team section.

Some intended outcomes from this assignment:

- You will individually get practice fitting a nonlinear regression models: polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and local regression

- You will practice using CV to select the optimal tuning parameters of your models
- You will use CV to compare the performance of these models
- You will make predictions based on your models
- You will practice fitting and interpreting a GAM
- You will gain experience working on a data science "project" as a team

Individual section (Jake)

Using the ILSR2 Auto dataset, I will regress miles per gallon (mpg) onto engine displacement (measured in cubic inches) using six non-linear models: polynomial regression, step function regression, cubic spline, natural spline, smoothing spline, and local regression.

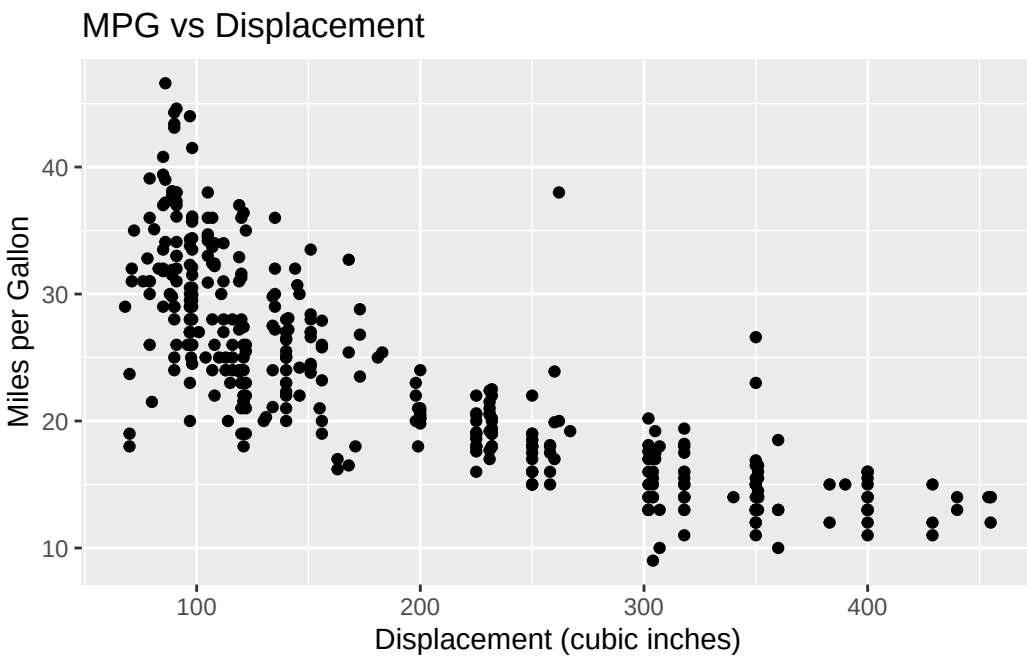
I will use LOOCV to choose hyperparameters for four methods: step function regression, cubic spline, natural spline, and smoothing spline. For each of these, I will choose the number of evaluate which number of breaks/knots yielding the lowest LOOCV MSE.

Fitting and evaluating all 6 models

```r
set.seed(0)
library(ISLR2)
library(boot)
library(splines)
library(ggplot2)
head(Auto)
```

```
  mpg cylinders displacement horsepower weight acceleration year origin
1  18         8          307        130   3504         12.0   70      1
2  15         8          350        165   3693         11.5   70      1
3  18         8          318        150   3436         11.0   70      1
4  16         8          304        150   3433         12.0   70      1
5  17         8          302        140   3449         10.5   70      1
6  15         8          429        198   4341         10.0   70      1
                       name
1 chevrolet chevelle malibu
2         buick skylark 320
3        plymouth satellite
4             amc rebel sst
5               ford torino
6          ford galaxie 500
```

```r
n <- nrow(Auto)

# plot mpg vs displacement ggplot
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  labs(title = "MPG vs Displacement", x = "Displacement (cubic inches)", y = "Miles per Gall
```



Clearly, the relationship is non-linear. To me, it looks like a negative logarithmic relationship. And, I think that 1-3 knots will be sufficient for a good spline fit.

Also, I think a polynomial fit should have at least degree 3 because neither a linear nor quadratic fit would capture this relationship well. For fair comparison, I will use degree 3 for both polynomial and natural spline fits.

Overall, based on the plot, I expect all methods to perform well expect step function regression because the rate of change of mpg, with respect to displacement, is very rapid for the first 50% of displacement values.

Polynomial regression

```r
# polynomial regression
poly_all <- glm(mpg ~ poly(displacement, degree = 3), data = Auto)
# loocv for polynomial regression
```

```r
cv_poly <- cv.glm(Auto, poly_all, K=n)
# get mse
mse_poly <- cv_poly$delta[1]
# plot model fit on all data
pred_poly <- predict(poly_all, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_poly), color = "blue") +
  labs(title = "Polynomial Regression Fit", x = "Displacement (cubic inches)", y = "Miles per
```

```r
print(paste("Polynomial Regression MSE:", round(mse_poly, 2)))
```

```
[1] "Polynomial Regression MSE: 19.19"
```

Polynomial regression with degree 3 had a great visual fit and the MSE was 19.19 under LOOCV.

Step function regression

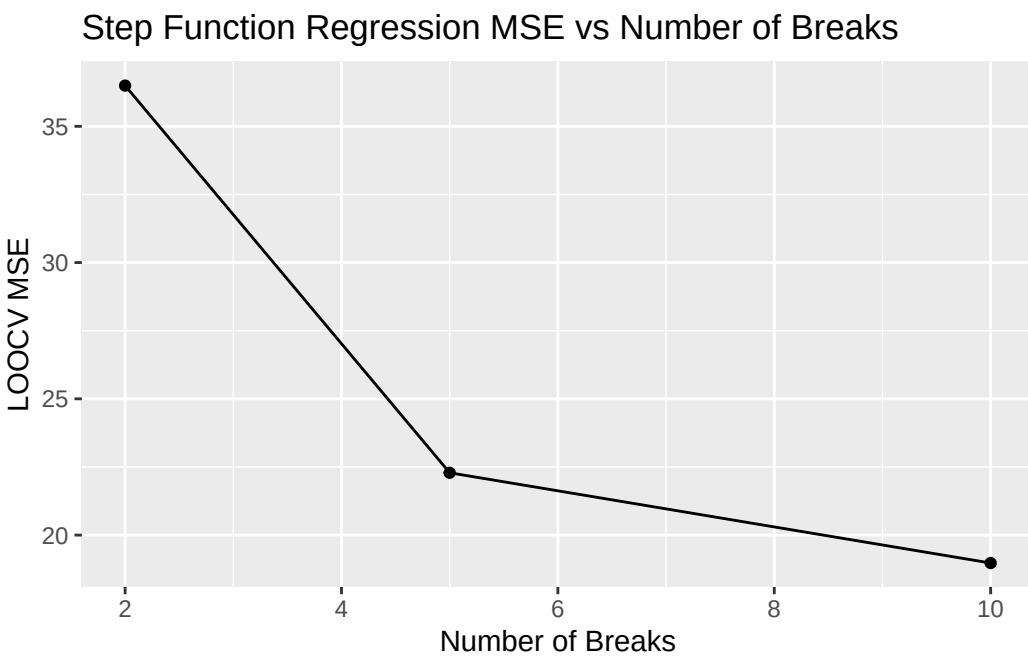
Given the visual relationship between displacement and mpg, I do not expect step function regression to perform well unless the number of breaks is very large. Therefore, I will try a

small, medium, and large number of breaks where I expect the best fit to be with a large number of breaks.

```
# breaks is number of different pieces
breaks <- c(2, 5, 10)
mse_step <- c()
# breakpoints need to be extended to handle errors with cut()
delta <- 0.1
for (b in breaks) {
  breakpoints <- seq(min(Auto$displacement) - delta, max(Auto$displacement) + delta, length.
  step_model <- glm(mpg ~ cut(displacement, breaks = breakpoints), data = Auto)
  cv_step <- cv.glm(Auto, step_model, K=n)
  mse_step <- c(mse_step, cv_step$delta[1])
  print(paste("Step function regression with", b, "breaks MSE:", round(cv_step$delta[1], 2)))
}
```

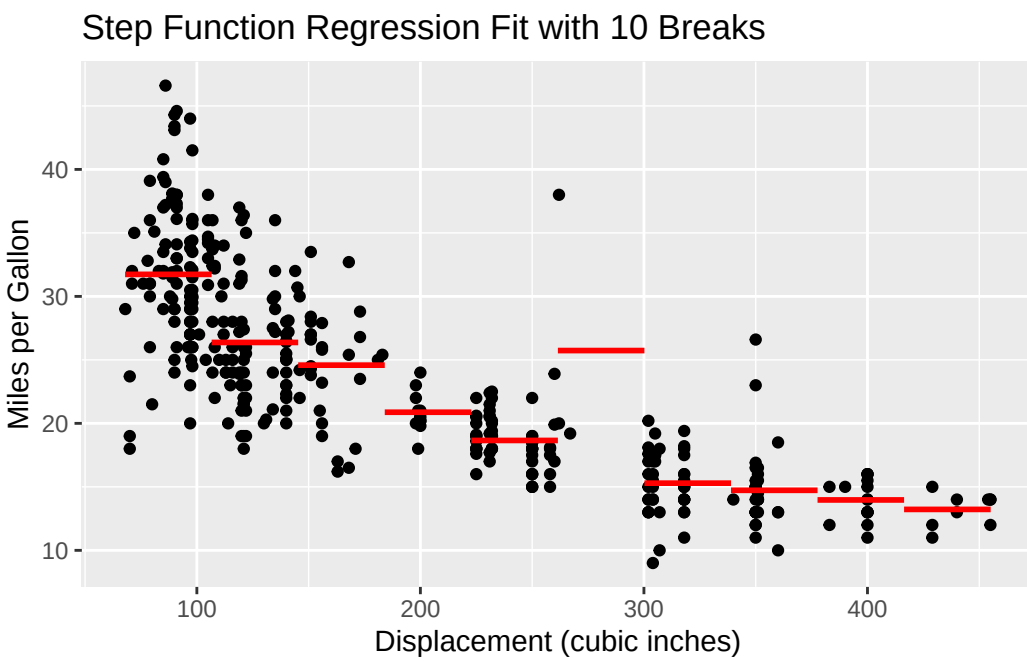
```
[1] "Step function regression with 2 breaks MSE: 36.49"
[1] "Step function regression with 5 breaks MSE: 22.29"
[1] "Step function regression with 10 breaks MSE: 18.98"
```

```
# plot breaks vs mse
ggplot(data.frame(breaks, mse_step), aes(x = breaks, y = mse_step)) +
  geom_point() +
  geom_line() +
  labs(title = "Step Function Regression MSE vs Number of Breaks", x = "Number of Breaks", y
```



As expected, MSE was minimized for the largest number of breaks (10) with MSE of 18.98.

```
# plot model fit for 10 breaks
breakpoints <- seq(min(Auto$displacement) - delta, max(Auto$displacement) + delta, length.out
step_model <- glm(mpg ~ cut(displacement, breaks = breakpoints), data = Auto)

# get the piecewise constant values for each interval
step_data <- data.frame(
  x_start = breakpoints[-length(breakpoints)],
  x_end = breakpoints[-1],
  y = coef(step_model)[1] + c(0, coef(step_model)[-1])
)

ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_segment(data = step_data, aes(x = x_start, xend = x_end, y = y, yend = y),
               color = "red", linewidth = 1) +
  labs(title = "Step Function Regression Fit with 10 Breaks", x = "Displacement (cubic inches
```



Clearly, the visual fit for step function regression is subpar compared to polynomial regression despite having better LOOCV MSE with 10 piecewise constants. LOOCV can underestimate error due to variance. For example, the outlier mpg point at ~ (260displacement, 37mpg) caused an undesierable fit for the piecewise constant in its interval.

Cubic spline

Given the visual relationship between displacement and mpg, I think few to no knots are needed for a good fit. Since 0 knots corresponds to a degree 3 polynomial fit, I will just try adding 1-3 knots and see if it improves the fit. I will place the knots at the quantiles of the displacement variable.

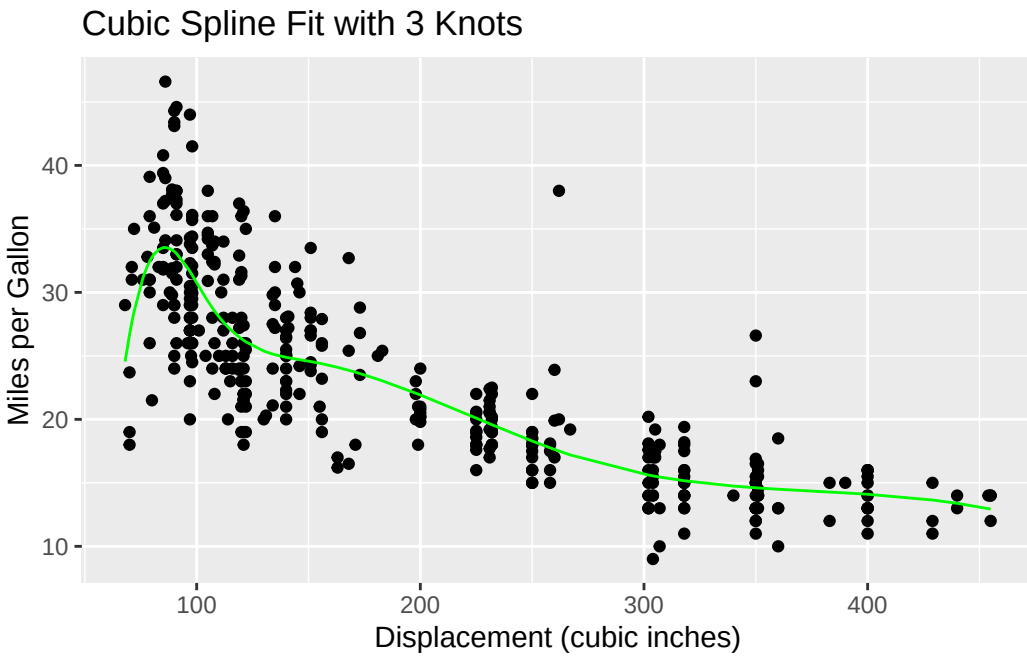
```
knots <- quantile(Auto$displacement, probs = c(0.25, 0.5, 0.75))
mse_cubic <- c()
for (k in 1:3) {
  cubic_model <- glm(mpg ~ bs(displacement, knots = knots[1:k], degree = 3), data = Auto)
  cv_cubic <- cv.glm(Auto, cubic_model, K=n)
  mse_cubic <- c(mse_cubic, cv_cubic$delta[1])
  print(paste("Cubic spline with", k, "knots MSE:", round(cv_cubic$delta[1], 2)))
}
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105), Boundary.knots
= c(70, : some 'x' values beyond boundary knots may cause ill-conditioned bases
```

```
Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105), Boundary.knots
= c(70, : some 'x' values beyond boundary knots may cause ill-conditioned bases

[1] "Cubic spline with 1 knots MSE: 18.61"

Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151:
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151:
some 'x' values beyond boundary knots may cause ill-conditioned bases

[1] "Cubic spline with 2 knots MSE: 17.75"

Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

Warning in bs(displacement, degree = 3L, knots = c(`25%` = 105, `50%` = 151, :
some 'x' values beyond boundary knots may cause ill-conditioned bases

[1] "Cubic spline with 3 knots MSE: 17.7"
```

Similar to step function regression, the most flexible cubic spline fit with 3 knots had the lowest MSE (17.7). The visual fit will reveal if overfitting is occurring.

```r
# plot model fit for 3 knots
cubic_model <- glm(mpg ~ bs(displacement, knots = knots[1:3], degree = 3), data = Auto)
pred_cubic <- predict(cubic_model, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_cubic), color = "green") +
  labs(title = "Cubic Spline Fit with 3 Knots", x = "Displacement (cubic inches)", y = "Miles
```



From the visual fit, the main difference between the cubic spline and polynomial fit is that the cubic spline fits a different leading tail that minimizes some more error for the smallest displacement values.

Natural spline

For natural splines, I will also try 1-3 knots at the same quantiles as the cubic spline. MSE will probably be minimized with a larger number of knots. Again, I will check the visual fit to see if overfitting is occurring.

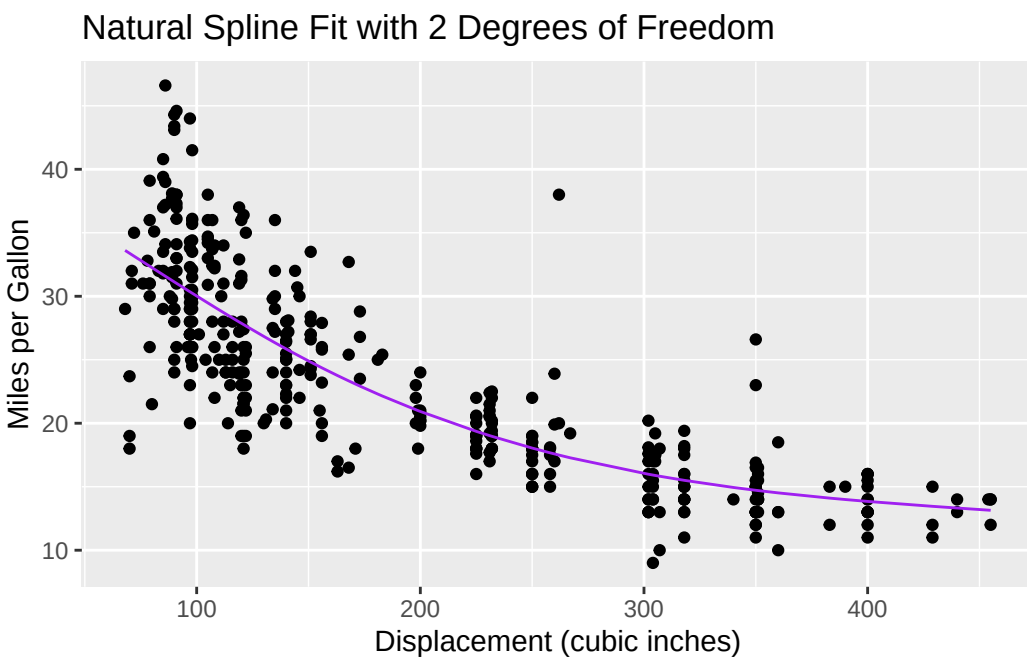
```
# natural spline uses degrees of freedom which is number of knots + 1
dfs <- c(2,3,4)

mse_natural <- c()
for (d in dfs) {
  natural_model <- glm(mpg ~ ns(displacement, df= d), data = Auto)
  cv_natural <- cv.glm(Auto, natural_model, K=n)
  mse_natural <- c(mse_natural, cv_natural$delta[1])
  print(paste("Natural spline with", d, "degrees of freedom MSE:", round(cv_natural$delta[1]
}
```

```
[1] "Natural spline with 2 degrees of freedom MSE: 19.1"
[1] "Natural spline with 3 degrees of freedom MSE: 19.2"
```

```
[1] "Natural spline with 4 degrees of freedom MSE: 19.36"
```

With degrees of freedom equal to 2 (1 knot) acheived the lowest MSE (19.1). I would guess the reason natural spline cannot overfit with more knots is because the linear tail constraint prevents the overfitting for small displacement values that we saw with the cubic spline fit.

```
# plot model fit for 2 degrees of freedom (1 knot)
natural_model <- glm(mpg ~ ns(displacement, df= 2), data = Auto)
pred_natural <- predict(natural_model, newdata = Auto)
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_natural), color = "purple") +
  labs(title = "Natural Spline Fit with 2 Degrees of Freedom", x = "Displacement (cubic inche
```



Smoothing spline

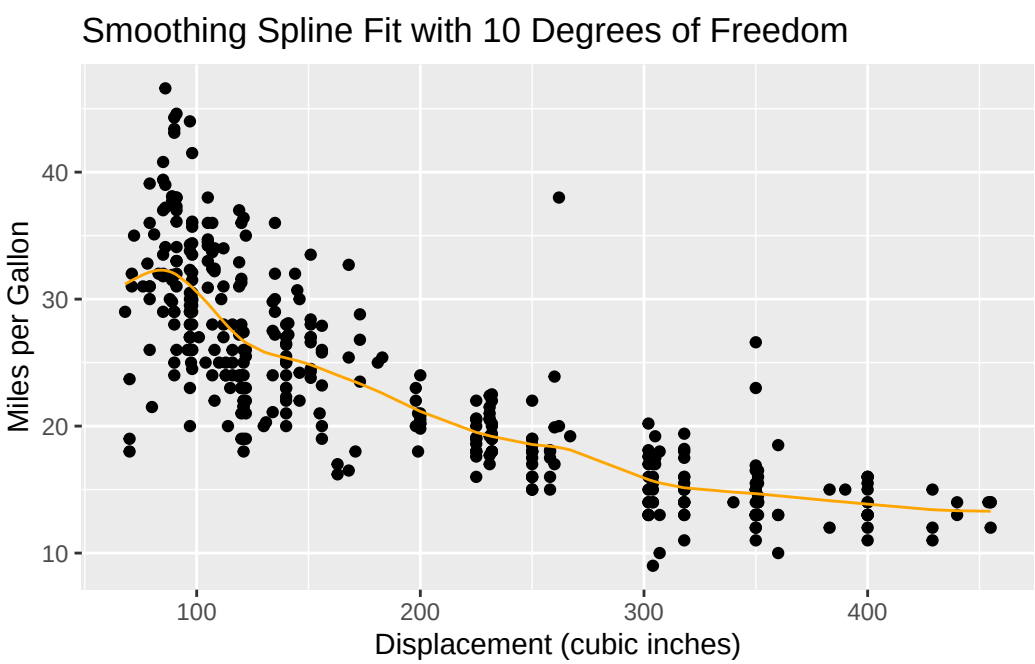
For hyperparameter tuning smoothing splines, the degrees of freedom do not correspond directly to the number of knots; instead, higher degrees of freedom just implies more smoothing flexibility. Therefore, I will try a small, medium, and large number of degrees of freedom to understand how the fit changes.

```r
dfs_smooth <- c(2,5,10)

mse_smooth <- c()
for (d in dfs_smooth) {
  smooth_model <- smooth.spline(Auto$displacement, Auto$mpg, df = d)
  pred_smooth <- predict(smooth_model, x = Auto$displacement)$y
  mse_smooth <- c(mse_smooth, mean((Auto$mpg - pred_smooth)^2))
  print(paste("Smoothing spline with", d, "degrees of freedom MSE:", round(mean((Auto$mpg - p
}
```

```
[1] "Smoothing spline with 2 degrees of freedom MSE: 21.35"
[1] "Smoothing spline with 5 degrees of freedom MSE: 18.71"
[1] "Smoothing spline with 10 degrees of freedom MSE: 17.6"
```

The minimum MSE was 17.6 for the most smooth spline (10 df).

```r
# plot model fit for 10 degrees of freedom
smooth_model <- smooth.spline(Auto$displacement, Auto$mpg, df = 10)
pred_smooth <- predict(smooth_model, x = Auto$displacement)$y
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_smooth), color = "orange") +
  labs(title = "Smoothing Spline Fit with 10 Degrees of Freedom", x = "Displacement (cubic in
```



The visual fit is an intermediate between the cubic spline and natural spline fits. The fit is more flexible than the natural spline fit, but the overfitting for small displacement values is less severe than the cubic.

Local regression

I expect local regression to perform similarly to cubic polynomial and natural spline regression since the visual relationship can probably be approximated by a sequence of linear fits.

I am not tuning the span hyperparameter. I will use the default.

```
local_model <- loess(mpg ~ displacement, data = Auto)
pred_local <- predict(local_model, newdata = Auto)
mse_local <- mean((Auto$mpg - pred_local)^2)
print(paste("Local regression MSE:", round(mse_local, 2)))
```

```
[1] "Local regression MSE: 18.86"
```

The MSE for local regression was 18.86.

```
# plot local regression fit
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_local), color = "brown") +
  labs(title = "Local Regression Fit", x = "Displacement (cubic inches)", y = "Miles per Gal
```

As expected, the local regression fit is similar to the cubic polynomial and natural spline fits.

Summary of 6 model evaluation

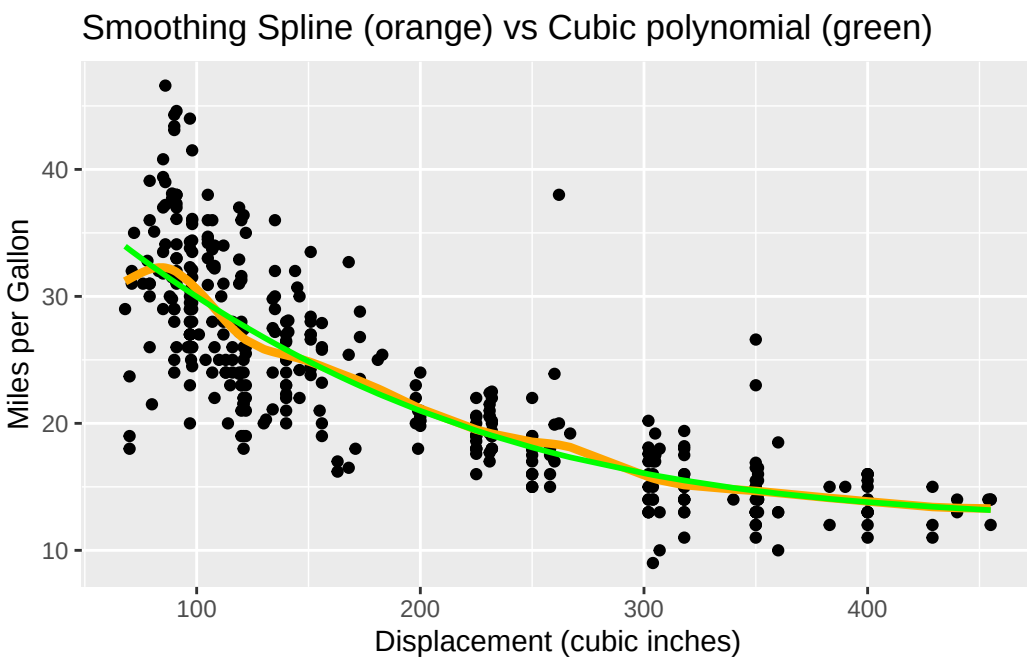
Across all 6 models and after hyperparameter tuning for 4 of the models, the lowest MSE was 17.6 for the smoothing spline with 10 degrees of freedom.

The instructions state to choose the best model from LOOCV MSE; however, I think the visualization of each model suggests that a simple cubic polynomial is probably the best option.

For completeness, I will plot both the smoothing spline and the cubic polynomial.

```
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_smooth), color = "orange", size = 1.5) +
  geom_line(aes(y = pred_poly), color = "green", size = 1) +
  labs(title = "Smoothing Spline (orange) vs Cubic polynomial (green)", x = "Displacement (cu
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
```



The smoothing spline is more flexible and has lower MSE, but the cubic polynomial is simpler and has a visually good fit.

Prediction at significant X

A significant value of displacement (X) is 260 cubic inches because there exists a high leverage point near this value in the training data set. This high leverage point had significant impact on the step function regression fit, and it is interesting to consider the tradeoff between choosing a model that fits this point well or not.

```
# predict at x0 = 260 for both cubic polynomial and smoothing spline
x0 <- 260
pred_poly_x0 <- predict(poly_all, newdata = data.frame(displacement = x0))
pred_smooth_x0 <- predict(smooth_model, x = x0)$y
print(paste("Prediction at x0 = 260 for cubic polynomial:", round(pred_poly_x0, 2)))
```

```
[1] "Prediction at x0 = 260 for cubic polynomial: 17.64"
```

```
print(paste("Prediction at x0 = 260 for smoothing spline:", round(pred_smooth_x0, 2)))
```

```
[1] "Prediction at x0 = 260 for smoothing spline: 18.37"
```

Interpretation

The predicted mpg for 260 cubic inches is about 18 mpg for both cubic polynomial and smoothing spline.

In conclusion, I think it is better to fit the majority of points nearby 260 cubic inches well rather than overfitting to the high leverage point at 260.

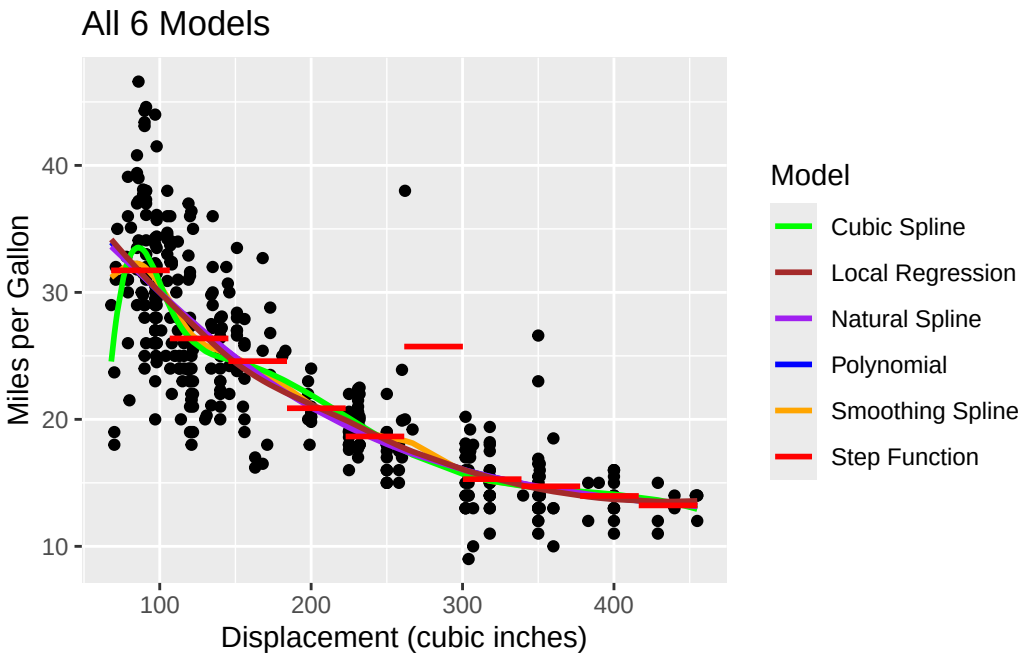
In general, the model shows that as engine displacement increases, miles per gallon decreases in a non-linear fashion.

Ethical implications

A potential ethical implication is that clearly displacement is not the only factor affecting miles per gallon. For an approximately fixed displacement value, there exists a range of mpgs in the training data. Therefore, this model is overly simplistic and this study is purely pedagogical. In practice, including more predictors of mpg would probably help produce a better model.

Final visualization of all 6 models

```
ggplot(Auto, aes(x = displacement, y = mpg)) +
  geom_point() +
  geom_line(aes(y = pred_poly, color = "Polynomial"), size = 1) +
  geom_line(aes(y = pred_smooth, color = "Smoothing Spline"), size = 1) +
  geom_line(aes(y = pred_cubic, color = "Cubic Spline"), size = 1) +
  geom_line(aes(y = pred_natural, color = "Natural Spline"), size = 1) +
  geom_line(aes(y = pred_local, color = "Local Regression"), size = 1) +
  geom_segment(data = step_data, aes(x = x_start, xend = x_end, y = y, yend = y, color = "Ste
               linewidth = 1) +
  scale_color_manual(values = c("Polynomial" = "blue",
                                "Smoothing Spline" = "orange",
                                "Cubic Spline" = "green",
                                "Natural Spline" = "purple",
                                "Local Regression" = "brown",
                                "Step Function" = "red")) +
  labs(title = "All 6 Models", x = "Displacement (cubic inches)", y = "Miles per Gallon", col
```


All 6 Models
Miles per Gallon
40
30
20
10
100
200
300
400
Displacement (cubic inches)
Model
Cubic Spline
Local Regression
Natural Spline
Polynomial
Smoothing Spline
Step Function